
Empty
CompareTo
Concat
EndsWith
String Methods
Replace
Equals

Java String

What is String in Java?

Generally, String is a sequence of characters. But in Java, string is an object that represents a sequence of characters. The java.lang.String class is used to create a string object.

How to create a string object?

There are two ways to create String object:

- **By string literal**
- **By new keyword**

1) String Literal

Java String literal is created by using double quotes. For Example:

```
String s="welcome";
```

2) By new keyword

```
String s=new String("Welcome");
//creates two objects and one reference variable
```

No.	Method	Description
1	char charAt(int index)	It returns char value for the particular index
2	int length()	It returns string length
3	static String format(String format, Object... args)	It returns a formatted string.
4	static String format(Locale l, String format, Object... args)	It returns formatted string with given locale.
5	String substring(int beginIndex)	It returns substring for given begin index.
6	String substring(int beginIndex, int endIndex)	It returns substring for given begin index and end index.
7	boolean contains(CharSequence s)	It returns true or false after matching the sequence of char value.

8	static String join(CharSequence delimiter, CharSequence... elements)	It returns a joined string.
9	static String join(CharSequence delimiter, Iterable<? extends CharSequence> elements)	It returns a joined string.
10	boolean equals(Object another)	It checks the equality of string with the given object.
11	boolean isEmpty()	It checks if string is empty.
12	String concat(String str)	It concatenates the specified string.
13	String replace(char old, char new)	It replaces all occurrences of the specified char value.
14	String replace(CharSequence old, CharSequence new)	It replaces all occurrences of the specified CharSequence.

length()

Returns the length of a string (the number of characters in the string).

```
String str = “VITAP";
System.out.println(str.length()); // Outputs 5
```

charAt(int index)

Returns the character at the specified index.

```
String str = “VITAP";
System.out.println(str.charAt(1));
// Outputs ‘I'
```

substring(int beginIndex, int endIndex)

Returns a substring from the specified beginIndex to endIndex-1.

```
String str = "Hello";
System.out.println(str.substring(1, 3));
// Outputs "el“
```

concat(String str)

Concatenates the specified string to the end of the calling string.

```
String str1 = “Vit";
String str2 = " Ap";
System.out.println(str1.concat(str2));
// Outputs “Vit Ap"
```

equals(Object anotherObject) :

Compares the calling string to the specified object. The result is true if and only if the argument is not null and is a String object that represents the same sequence of characters as this object.

```
String str1 = "Hello";
String str2 = "Hello";
System.out.println(str1.equals(str2)); // Outputs true
```

equalsIgnoreCase(String anotherString):

Compares the calling string to another string, ignoring case considerations.

```
String str1 = "hello";
String str2 = "HELLO";
System.out.println(str1.equalsIgnoreCase(str2)); // Outputs true
```

toLowerCase() and toUpperCase()

Converts all the characters in the string to lower case or upper case.

```
String str = "Hello World";
System.out.println(str.toLowerCase()); // Outputs "hello world"
System.out.println(str.toUpperCase()); // Outputs "HELLO WORLD“
```

trim()

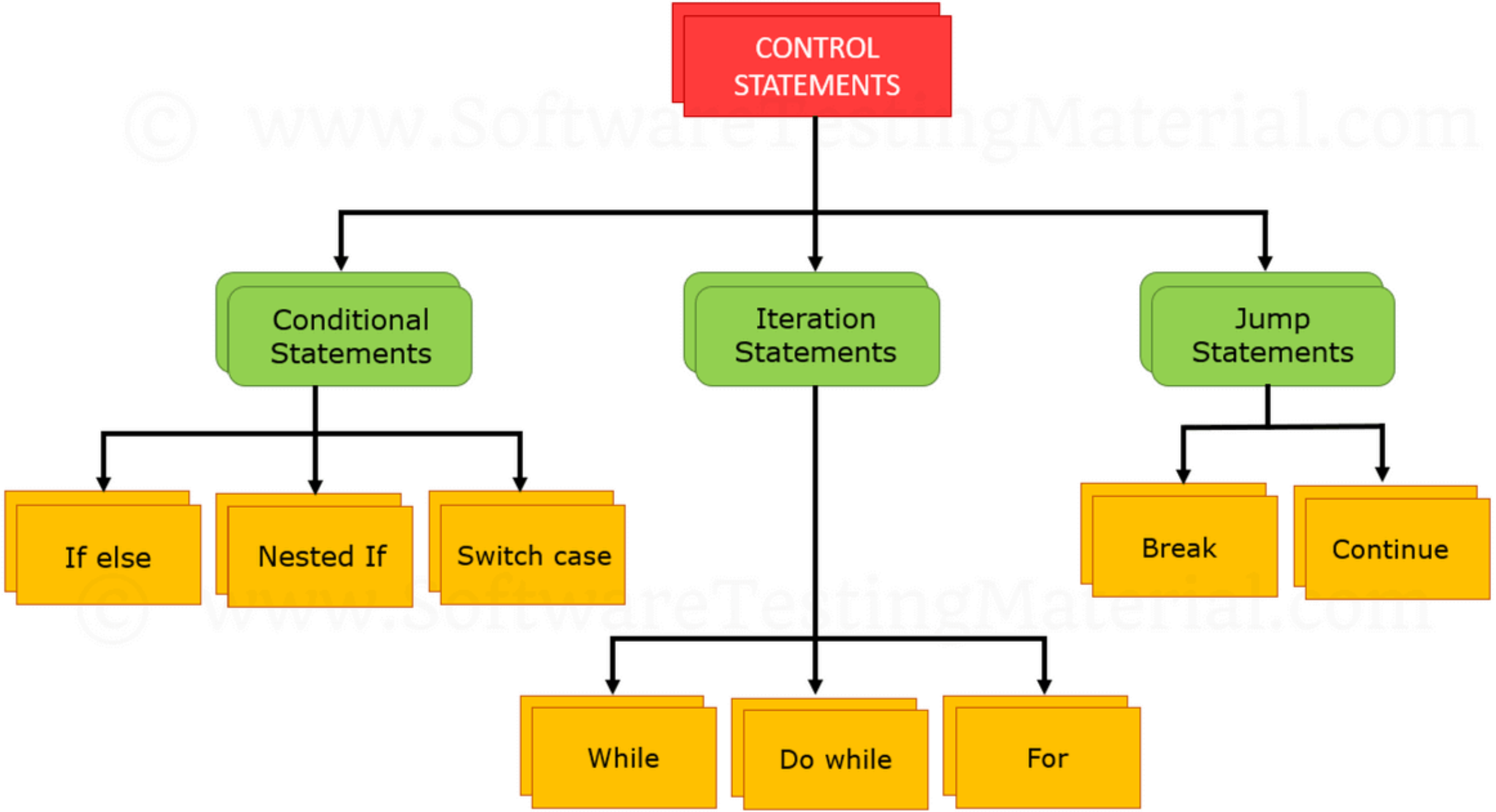
Returns a copy of the string, with leading and trailing whitespace omitted.

```
String str = "  Hello World  ";
System.out.println(str.trim()); // Outputs "Hello World"
```

replace(char oldChar, char newChar)

Returns a new string resulting from replacing all occurrences of oldChar in this string with newChar.

```
String str = "Hello World";
System.out.println(str.replace('l', 'p')); // Outputs "Heppo Worpdp"
```



Decision Making Statements

if statement
if-else statement
switch statement

Loop Statements

while loop
do-while loop
for loop
for-each loop

Branching Statements

break
continue
return

If-else Statement

The Java ***if statement*** is used to test the condition. It checks boolean condition: ***true*** or ***false***. There are various types of if statement in java.

- if statement
- if-else statement
- if-else-if ladder
- nested if statement

Java if Statement

The Java if statement tests the condition. It executes the ***if block*** if condition is true.

Syntax:

```
1. if(condition){
2. //code to be executed
3. }
```

if statement

```
if (condition) {
    // block of code to be executed if the condition is true
}
```

if: This keyword checks the condition given in parentheses.

condition: A boolean expression that, when true, executes the block of code inside the curly braces.

Block of code: If the condition is true, the statements inside the curly braces are executed.

Example:

```
1. //Java Program to demonstate the use of if statement.
2. public class IfExample {
3. public static void main(String[] args) {
4.     //defining an 'age' variable
5.     int age=20;
6.     //checking the age
7.     if(age>18){
8.         System.out.print("Age is greater than 18");
9.     }
10. }
11. }
```

Output:

```
Age is greater than 18
```

Java if-else Statement

The Java if-else statement also tests the condition. It executes the ***if block*** if condition true otherwise ***else block*** is executed.

Syntax:

```
1. if(condition){
2. //code if condition is true
3. }else{
4. //code if condition is false
5. }
```

```java
// Define a class named 'NumberCheck'
public class NumberCheck
{
    // The main method - entry point of the Java application
    public static void main(String[] args) {

        // Declare an int variable 'number' and initialize it with -10
        int number = -10;

        // An 'if' statement to check if the variable 'number' is greater than 0
        if (number > 0) {
            // This line is executed only if the 'if' condition is true
            // Prints "The number is positive." to the console
            System.out.println("The number is positive.");
             }
         else {
            // This line is executed only if the 'if' condition is false
            // Prints "The number is not positive." to the console
            System.out.println("The number is not positive.");
        }
    }
}
```

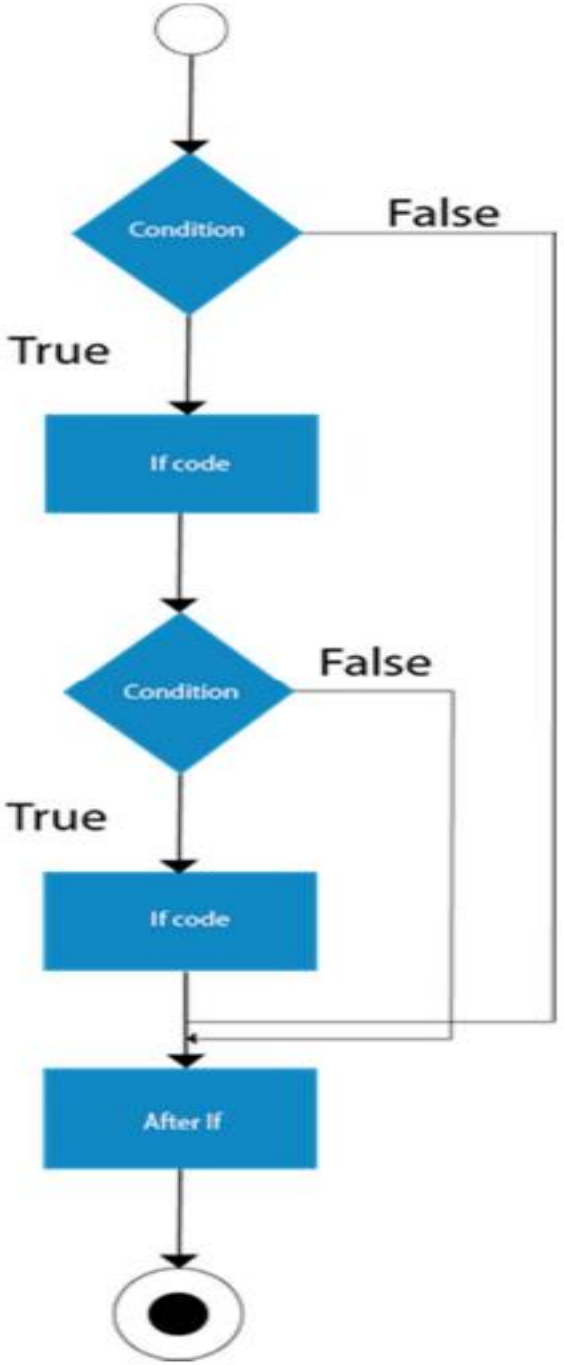
```
public class Test {

   public static void main(String args[]) {
      int x = 30;

      if( x == 10 ) {
         System.out.print("Value of X is 10");
      }else if( x == 20 ) {
         System.out.print("Value of X is 20");
      }else if( x == 30 ) {
         System.out.print("Value of X is 30");
      }else {
         System.out.print("This is else statement");
      }
   }
}
```

Java Nested if statement

The nested if statement represents the ***if block within another if block***. Here, the inner if block condition executes only when outer if block condition is true.

Syntax:

```
1. if(condition){
2.     //code to be executed
3.         if(condition){
4.             //code to be executed
5.     }
6. }
```




```java
//Java Program to demonstrate the use of Nested If Statement.
public class JavaNestedIfExample {
public static void main(String[] args) {
    //Creating two variables for age and weight
    int age=20;
    int weight=80;
    //applying condition on age and weight
    if(age>=18){
        if(weight>50){
            System.out.println("You are eligible to donate blood");
        }
    }
}}
```

Output:

```
You are eligible to donate blood
```

Example 2:

```
1. //Java Program to demonstrate the use of Nested If Statement.
2. public class JavaNestedIfExample2 {
3. public static void main(String[] args) {
4.     //Creating two variables for age and weight
5.     int age=25;
6.     int weight=48;
7.     //applying condition on age and weight
8.     if(age>=18){
9.         if(weight>50){
10.            System.out.println("You are eligible to donate blood");
11.        } else{
12.            System.out.println("You are not eligible to donate blood");
13.        }
14.    } else{
15.      System.out.println("Age must be greater than 18");
16.    }
17.} }
```

Output:

```
You are not eligible to donate blood
```

Java if-else-if ladder Statement

The if-else-if ladder statement executes one condition from multiple statements.

Syntax:

```
if(condition1){
//code to be executed if condition1 is true
}else if(condition2){
//code to be executed if condition2 is true
}
else if(condition3){
//code to be executed if condition3 is true
}
...
else{
//code to be executed if all the conditions are false
}
```

```
2. //It is a program of grading system for fail, D grade, C grade, B grade, A grade and A+.
3. public class IfElseIfExample {
4. public static void main(String[] args) {
5.     int marks=65;
6.
7.     if(marks<50){
8.         System.out.println("fail");
9.     }
10.    else if(marks>=50 && marks<60){
11.        System.out.println("D grade");
12.    }
13.    else if(marks>=60 && marks<70){
14.        System.out.println("C grade");
15.    }
16.    else if(marks>=70 && marks<80){
17.        System.out.println("B grade");
18.    }
19.    else if(marks>=80 && marks<90){
20.        System.out.println("A grade");
21.    }else if(marks>=90 && marks<100){
22.        System.out.println("A+ grade");
23.    }else{
24.        System.out.println("Invalid!");
25.    }
26.}
27.}
```

Output:

```
C grade
```

A school has following rules for grading system:

a. Below 25 - F

b. 25 to 45 - E

c. 45 to 50 - D

d. 50 to 60 - C

e. 60 to 80 - B

f. Above 80 - A

Ask user to enter marks and print the corresponding grade.

Java Switch Statement

The Java ***switch statement*** executes one statement from multiple conditions. It is like if-else-if ladder statement. The switch statement works with byte, short, int, long, enum types, String and some wrapper types like Byte, Short, Int, and Long. Since Java 7, you can use strings in the switch statement.

In other words, the switch statement tests the equality of a variable against multiple values.

Syntax:

```
1. switch(expression){
2. case value1:
3.  //code to be executed;
4.  break;  //optional
5. case value2:
6.  //code to be executed;
7.  break;  //optional
8. ......
9.
10. default:
11. code to be executed if all cases are not matched;
12. }
```

```
1.  public class SwitchExample {
2.  public static void main(String[] args) {
3.      //Declaring a variable for switch expression
4.      int number=20;
5.      //Switch expression
6.      switch(number){
7.      //Case statements
8.      case 10: System.out.println("10");
9.      break;
10.     case 20: System.out.println("20");
11.     break;
12.    case 30: System.out.println("30");
13.    break;
14.    //Default case statement
15.    default:System.out.println("Not in 10, 20 or 30");
16.    }
17.}
18.}
```

Loops in Java

In programming languages, loops are used to execute a set of instructions/functions repeatedly when some conditions become true. There are three types of loops in java.

- for loop
- while loop
- do-while loop

Java Simple For Loop

A simple for loop is the same as C/C++. We can initialize the variable, check condition and increment/decrement value. It consists of four parts:

1. **Initialization**: It is the initial condition which is executed once when the loop starts. Here, we can initialize the variable, or we can use an already initialized variable. It is an optional condition.
2. **Condition**: It is the second condition which is executed each time to test the condition of the loop. It continues execution until the condition is false. It must return boolean value either true or false. It is an optional condition.
3. **Statement**: The statement of the loop is executed each time until the second condition is false.
4. **Increment/Decrement**: It increments or decrements the variable value. It is an optional condition.

Syntax:

```
1. for(initialization;condition;incr/decr){
2. //statement or code to be executed
3. }
```

for (int i = 1; i <= 5; i++)

Java Labeled For Loop

Syntax:

```
1. labelname:
2. for(initialization;condition;incr/decr){
3. //code to be executed
4. }
```

for-each loop

The for-each loop is used to loop through elements in an array or a collection.

```
int[] numbers = {1, 2, 3, 4, 5};
for (int number : numbers) {
    System.out.println(number);
}
```

Java While Loop

The Java ***while loop*** is used to iterate a part of the program several times. If the number of iteration is not fixed, it is recommended to use while loop.

Syntax:

```
1. while(condition){
2. //code to be executed
3. }
```

```
1.  public class WhileExample {
2.  public static void main(String[] args) {
3.      int i=1;
4.      while(i<=10){
5.          System.out.println(i);
6.      i++;
7.      }
8.  }
9.  }
```

```
1
2
3
4
5
6
7
8
9
10
```

Java do-while Loop

The Java ***do-while loop*** is used to iterate a part of the program several times. If the number of iteration is not fixed and you must have to execute the loop at least once, it is recommended to use do-while loop.

The Java ***do-while loop*** is executed at least once because condition is checked after loop body.

Syntax:

```
1. do{
2. //code to be executed
3. }while(condition);
```

```
1.  public class DoWhileExample {
2.  public static void main(String[] args) {
3.      int i=1;
4.      do{
5.          System.out.println(i);
6.      i++;
7.      }while(i<=10);
8.  }
9.  }
```

```
1
2
3
4
5
6
7
8
9
10
```

Branching Statements

break

The break statement is used to exit a loop or a switch statement.

```
for (int i = 1; i <= 10; i++)
 {
   if (i == 6)
   {
      break; // It will exit the loop when i is 6
   }
   System.out.println(i);
}
```

Continue

The continue statement skips the current iteration of a loop and continues with the next iteration.

```
for (int i = 1; i <= 10; i++)
{
  if (i % 2 == 0)
  {
    continue; // It will skip the rest of the loop body if i is even
  }
  System.out.println(i);
}
```

```
1
3
5
7
9
```

continue;: The continue statement is executed when i is even. It causes the loop to immediately stop executing the remaining code in the current iteration and proceed with the next iteration of the loop.

Return

The return statement exits from the current method and optionally returns a value.

```
int square(int number)
{
   return number * number;
}
int result = square(5);
System.out.println(result); // This will print 25
```

Java Program to demonstrate the example of Switch statement

Print the month name for the given number

for loop

A five-digit number ABCDE is called Lucky if A+B+C=D+E

Write a Java program that asks the user to enter a five digit number and tell if the number is a Lucky number or not

Ex: 13857 is lucky number

98712 is not a lucky number

While and Do....

Write a java program that reads a number between 1 and 100 from the user

Sample output:

Enter a number:

130

Sorry , 130 is not between 1 and 100

Enter a number:1000

Sorry , 1000 is not between 1 and 100

Enter a number: 50

50 is between 1 and 100

Thank you .....Welcome again

Write a calculator program using Switch case and do...while statement.

Example:
(Press '+' for Addition, '-' for Subraction, '*' for Multiplication and '/' for division,
Enter the operator: +
 Enter the first number:12
Enter the Second number:20
The result is 32
 Do you want to continue(Y/N): Y
 Enter the operator: *
 Enter the first number:5
Enter the Second number:10
The result is 50
Do you want to continue(Y/N): N
 Bye.....